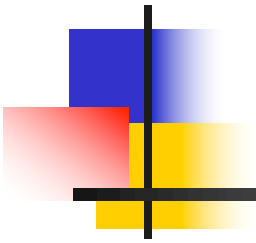
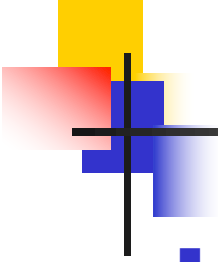


Tema 2: Capa Acceso a Datos con Spring y JPA

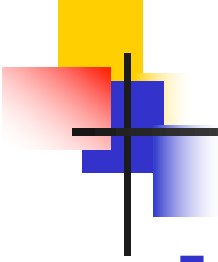


- Introducción
- Spring Boot
- Diagrama de entidades en PA Shop
- Modelado básico de entidades
- Modelado de relaciones
- Métodos de negocio en entidades
- Desarrollo de DAOs



Introducción

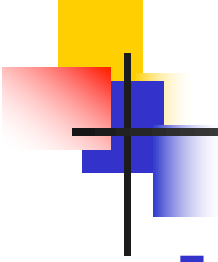
- Para implementar la capa de Acceso a Datos usaremos JPA/Hibernate y Spring Data
- JPA (Java Persistence API)
 - <https://jakarta.ee/specifications/persistence/3.0>
 - API estándar de un mapeador objeto-relacional
 - Paquete principal `jakarta.persistence`
 - Proporciona anotaciones para mapear entidades a una base de datos relacional
 - Y una API para interactuar con la base de datos (e.g. operaciones CRUD, lenguaje de consultas, etc.)
 - Internamente, las implementaciones de JPA hacen uso de la API de JDBC
- Hibernate ORM
 - <https://hibernate.org/orm>
 - Libro ***Java Persistence with Hibernate*** (bibliografía recomendada)
 - Implementación popular de JPA



Introducción

- Spring Data

- <https://spring.io/projects/spring-data>
- Librería para implementar ágilmente DAOs (repositorios, en terminología de Spring) sobre diversas fuentes de datos (BD relacionales, NoSQL, LDAP, servicios REST, etc.)
- En particular, usaremos Spring Data para JPA
 - <https://spring.io/projects/spring-data-jpa>



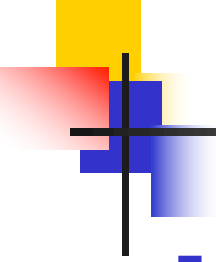
Introducción

- Tablas

- Hibernate permite crearlas automáticamente a partir de las entidades: opción válida para un prototipo o el arranque de un proyecto
- O adaptarse a las tablas que existan: opción más realista
 - E.g. un administrador de BD puede crear y mantener las tablas de forma más elaborada (e.g. con índices específicos, siguiendo convenciones de nombrado de la empresa, etc.)

- Mapping de entidades a tablas

- Con JPA se puede hacer mediante anotaciones sobre las entidades y/o ficheros de configuración



Introducción

- PA Shop

- Tablas

- Script: `backend/src/sql/1-MySQLCreateTables.sql`

- Creación de las tablas en la BD `pa`

- `cd backend`

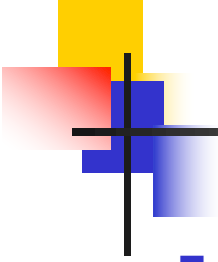
- `mvn sql:execute`

- Creación de las tablas en la BD `patest`

- Automáticamente cada vez que se ejecuta `mvn test`

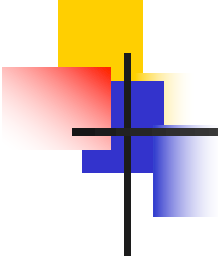
- Mapping de entidades a tablas

- Se usan anotaciones



Spring Boot

- Para el desarrollo de las distintas capas que conforman el backend haremos uso de Spring Boot
 - <https://spring.io/projects/spring-boot>
 - Libro *Spring Boot in Action* (bibliografía recomendada)
- “Starter poms”
 - Un “starter pom” es un `pom.xml` que incluye todas las dependencias necesarias para hacer uso de una característica concreta que necesita la aplicación (e.g. capa Modelo con JPA, aplicación/servicio web, etc.)
- Configura automáticamente las librerías (e.g. Spring Data, Hibernate, Spring MVC, etc.)
 - La configuración se puede adaptar a las necesidades del proyecto mediante ficheros de configuración `application.{properties,yml}`



Spring boot - starter poms - pom.xml

```
<dependencies>
```

Incluye las dependencias de Hibernate y Spring Data

```
  <dependency>
```

```
    <groupId>org.springframework.boot</groupId>
```

```
    <artifactId>spring-boot-starter-data-jpa</artifactId>
```

```
  </dependency>
```

```
  <dependency>
```

```
    <groupId>org.springframework.boot</groupId>
```

```
    <artifactId>spring-boot-starter-test</artifactId>
```

```
    <scope>test</scope>
```

Incluye las dependencias necesarias para
desarrollar pruebas automatizadas con JUnit

```
  </dependency>
```

```
  <dependency>
```

```
    <groupId>${jdbcDriver.groupId}</groupId>
```

```
    <artifactId>${jdbcDriver.artifactId}</artifactId>
```

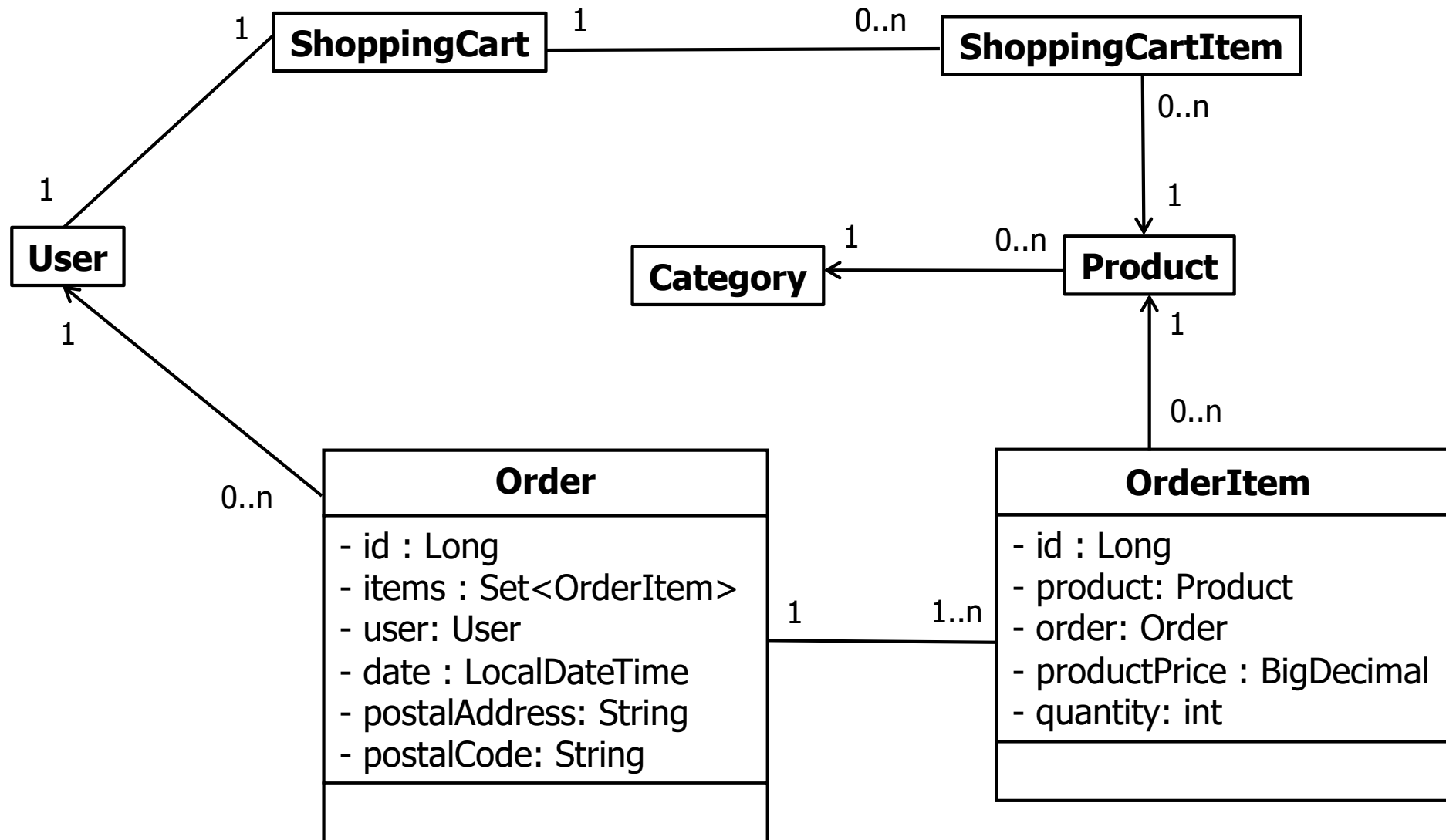
```
    <scope>runtime</scope>
```

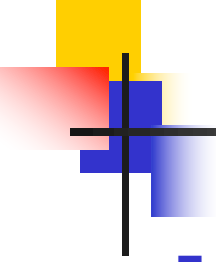
```
  </dependency>
```

```
  ...
```

```
</dependencies>
```

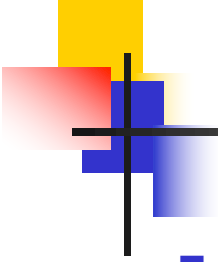
Diagrama de entidades en PA Shop





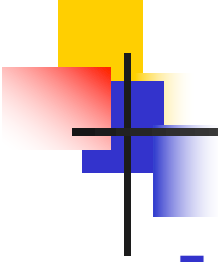
Modelado básico de entidades

- Requisitos de una clase entidad
 - Se anota con `@Entity`
 - NOTA: las anotaciones de JPA están definidas en `jakarta.persistence`
 - Debe tener un constructor sin argumentos público o protegido
 - No debe ser `final`
 - Debe tener una clave primaria
 - Puede tener relaciones de herencia (no lo estudiaremos) o de asociación con otras (lo estudiaremos más adelante)
 - Estado persistente: conjunto de atributos que se mapearán a columnas



Modelado básico de entidades

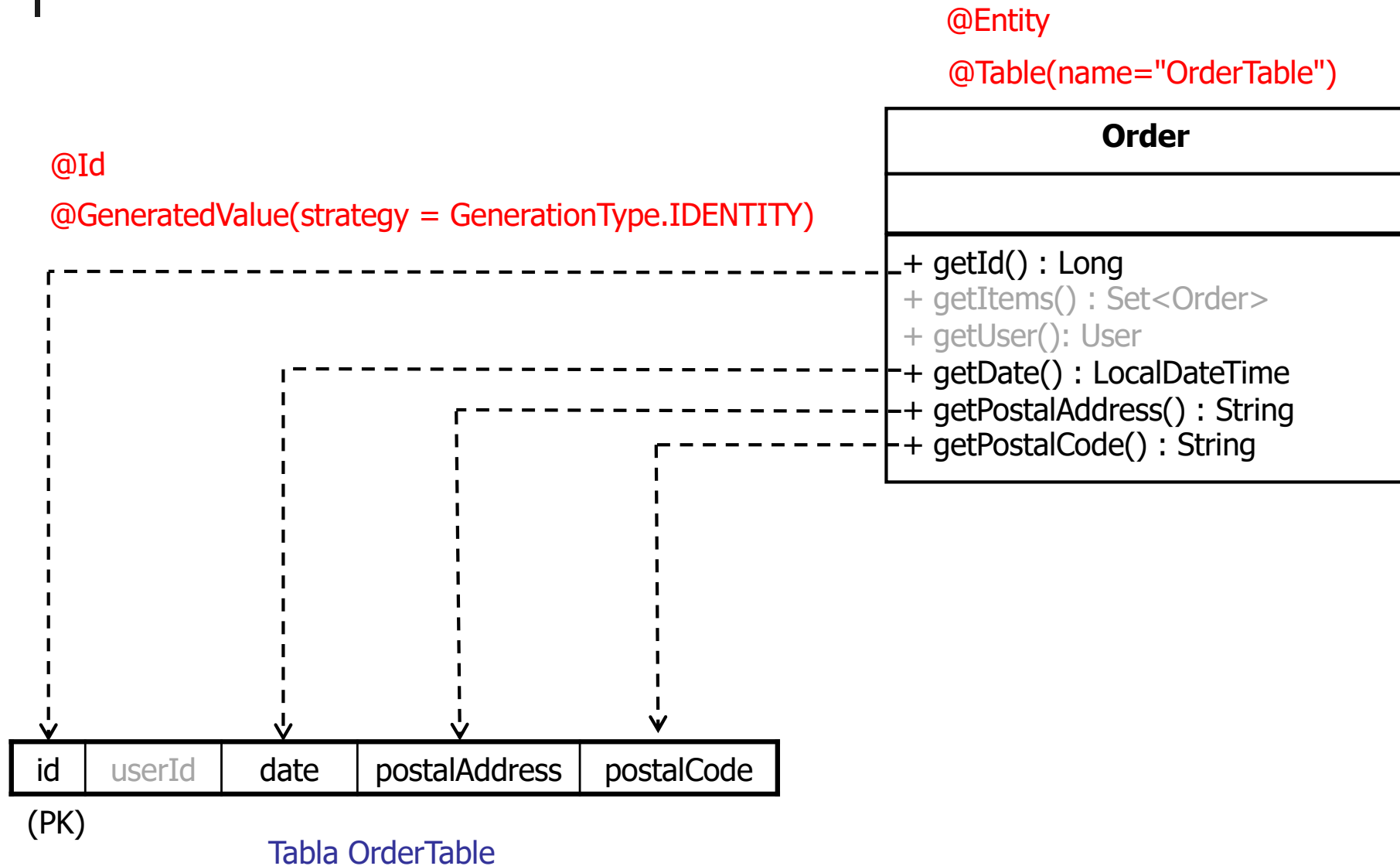
- Estado persistente: tipos soportados en los atributos
 - Tipos primitivos y sus contrapartidas objetuales: se mapean a tipos SQL equivalentes
 - `java.lang.String`, `java.math.BigInteger`, `java.math.BigDecimal`, `byte[]`, `Byte[]`, `char[]` y `Character[]`: se mapean a tipos SQL equivalentes
 - Fechas: se mapean a tipos SQL equivalentes
 - `java.time.LocalDate`, `java.time.LocalTime`, `java.time.LocalDateTime`
 - `java.util.Date`, `java.util.Calendar`
 - Enumerados: por defecto, se mapean a un tipo SQL numérico
 - Otras entidades y colecciones de entidades: relaciones (lo estudiaremos más adelante)

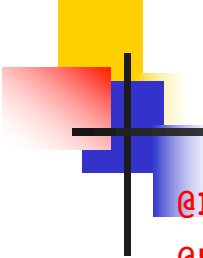


Modelado básico de entidades

- Estado persistente: acceso
 - El mapeador objeto-relacional puede acceder al estado persistente a través de los atributos (tipo de acceso campo) XOR mediante métodos **get/set** (tipo de acceso propiedad)
 - NOTA: en el caso del tipo de acceso propiedad, si la propiedad es booleana, el método de lectura puede llamarse **get<<Propiedad>>** o **is<<Propiedad>>**
 - Nosotros usaremos tipo de acceso propiedad
 - En caso de usar anotaciones, éstas han de colocarse sobre los métodos **get/is**

Modelado básico de entidades: Order





Modelado básico de entidades: Order

```
@Entity
```

```
@Table(name="OrderTable")
```

```
public class Order {
```

```
...
```

```
private Long id;
```

```
private Set<OrderItem> items = new HashSet<>();
```

```
private User user;
```

```
private LocalDateTime date;
```

```
private String postalAddress;
```

```
private String postalCode;
```

```
public Order() {}
```

```
public Order(User user, LocalDateTime date, String postalAddress,
```

```
String postalCode) {
```

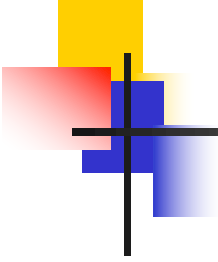
```
    this.user = user;
```

```
    this.date = date;
```

```
    this.postalAddress = postalAddress;
```

```
    this.postalCode = postalCode;
```

```
}
```



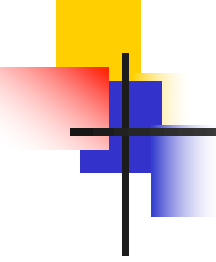
Modelado básico de entidades: Order

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
public Long getId() {
    return id;
}

public void setId(Long id) {
    this.id = id;
}

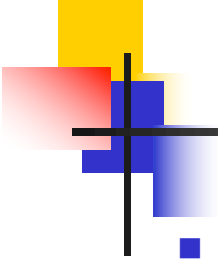
public LocalDateTime getDate() {
    return date;
}

public void setDate(LocalDateTime date) {
    this.date = date;
}
```



Modelado básico de entidades: Order

```
    public String getPostalAddress() {  
        return postalAddress;  
    }  
  
    public void setPostalAddress(String postalAddress) {  
        this.postalAddress = postalAddress;  
    }  
  
    public String getPostalCode() {  
        return postalCode;  
    }  
  
    public void setPostalCode(String postalCode) {  
        this.postalCode = postalCode;  
    }  
  
    ...  
}
```



Modelado básico de entidades

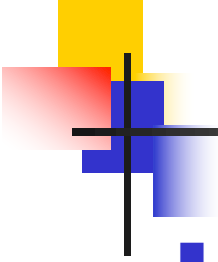
■ @Table

- Por defecto, una entidad se mapea a una tabla con el mismo nombre que el nombre simple de la clase
- Cuando el nombre de la tabla es distinto, hay que usar **@Table**
- En el ejemplo, la tabla en la que se mapea la entidad **Order** se llama **OrderTable**
 - NOTA: se hace así para evitar usar **order** como nombre de tabla, dado que coincide con una palabra reservada en SQL

■ @Column

- Por defecto, cada atributo/propiedad persistente se mapea a una columna con el mismo nombre
- Cuando el nombre de la columna sea distinto, hay que usar **@Column**

```
@Column(name="address")  
public String getPostalAddress() {  
    return postalAddress;  
}
```



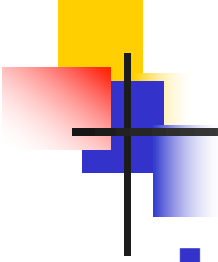
Modelado básico de entidades

- **@Id**

- Especifica el atributo/propiedad clave
- Sólo vale para claves simples
- Existen anotaciones para claves compuestas

- **@GeneratedValue**

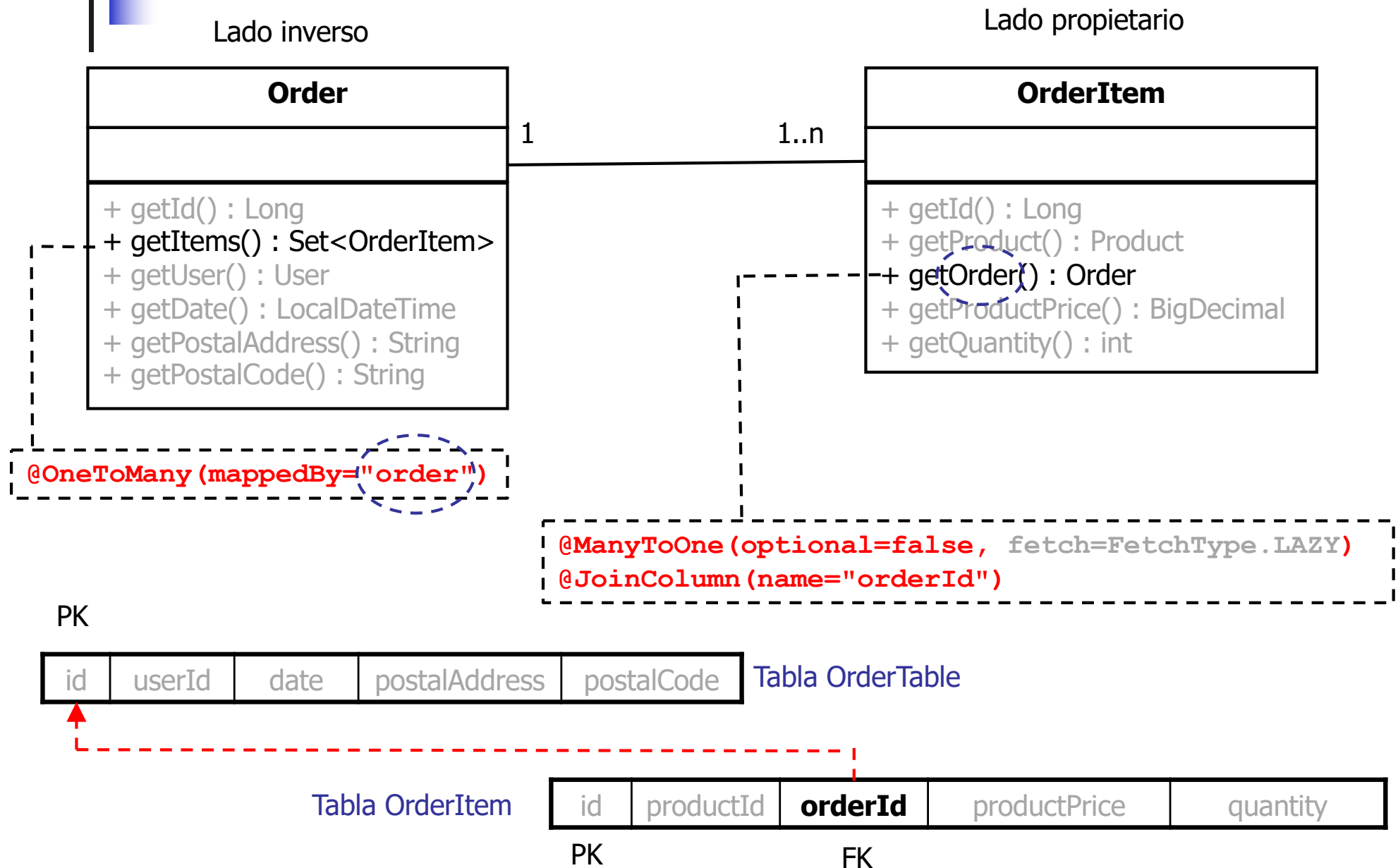
- Se puede usar conjuntamente con **@Id** si la clave es numérica y se desea que ésta se genere automáticamente
- En el ejemplo se usa la estrategia **GenerationType.IDENTITY**, que asume que la tabla usa una columna contador para la clave (e.g. MySQL)
- Si se usase una secuencia (e.g. Oracle), se debería especificar la estrategia **GenerationType.SEQUENCE**, seguramente en conjunción con la anotación **@SequenceGenerator**

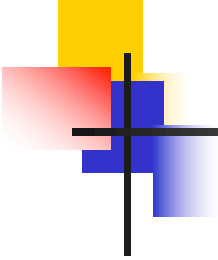


Modelado de relaciones

- Se pueden modelar relaciones 1:1, 1:N y N:M, unidireccionales o bidireccionales
 - Una relación es bidireccional cuando se puede navegar de una entidad a la otra, y de la segunda a la primera
 - En nuestro caso (tipo de acceso propiedad), mediante métodos `get/set`
- No estudiaremos las relaciones N:M
 - En general, no se considera una buena práctica usarlas
 - Alternativamente, se pueden modelar con dos relaciones 1:N y una entidad intermedia
 - Permite añadir atributos propios de la relación, ahora y/o en el futuro
- En una relación entre dos entidades
 - Una actúa como lado propietario
 - La entidad cuya tabla asociada tiene la clave foránea que mantiene la relación
 - Y otra como lado inverso
 - Sólo existe en el caso de las relaciones bidireccionales

Modelado de relaciones 1:N - Order-OrderItem





Modelado de relaciones 1:N - Order-OrderItem

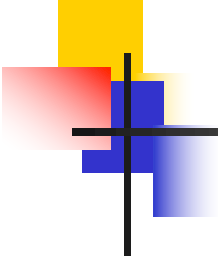
```
@Entity
@Table(name="OrderTable")
public class Order {

    ...
    private Set<OrderItem> items = new HashSet<>();

    @OneToMany(mappedBy="order")
    public Set<OrderItem> getItems() {
        return items;
    }

    public void setItems(Set<OrderItem> items) {
        this.items = items;
    }

    ...
}
```



Modelado de relaciones 1:N - Order-OrderItem

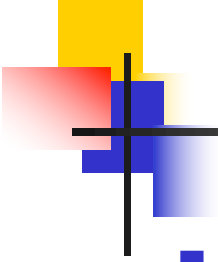
```
@Entity
public class OrderItem {

    ...
    private Order order;

    @ManyToOne(optional=false, fetch=FetchType.LAZY)
    @JoinColumn(name="orderId")
    public Order getOrder() {
        return order;
    }

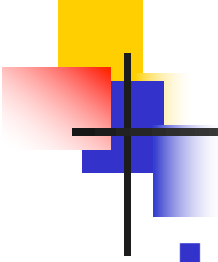
    public void setOrder(Order order) {
        this.order = order;
    }

    ...
}
```



Modelado de relaciones 1:N

- Se anotan los métodos **get** que permiten navegar de una entidad a la otra
- Las anotaciones **@ManyToOne** y **@OneToMany** son consistentes con la notación de cardinalidad en UML
- **@ManyToOne** se coloca sobre el método **get** de la entidad con cardinalidad Muchos, que permite navegar a la entidad con cardinalidad Uno
- Y de forma similar, **@OneToMany**
 - En general, el método **get** usa **Set** para devolver la colección de entidades relacionadas, porque normalmente no hay un orden predefinido en la relación



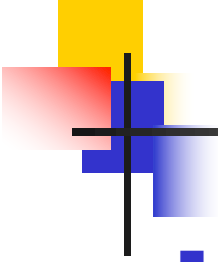
Modelado de relaciones 1:N

- **mappedBy**

- Sólo con relaciones bidireccionales
- Se utiliza en el lado inverso
- Especifica el nombre del atributo/propiedad del otro lado (lado propietario) de la relación
 - El nombre de una propiedad es el nombre del método `get`, sin el prefijo `get` y con la primera letra en minúscula
 - E.g. Método `getOrder` en `OrderItem` => propiedad: `order`

- **@JoinColumn**

- Se coloca sobre el método `get` de la entidad propietaria de la relación
- En una relación 1:N, el lado propietario siempre es el lado N
- Especifica el nombre de la columna que actúa como clave foránea



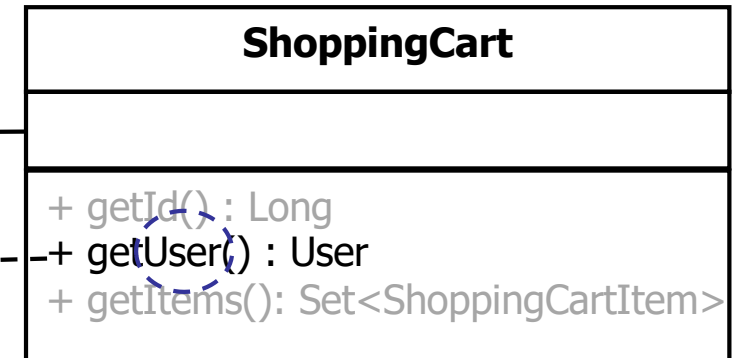
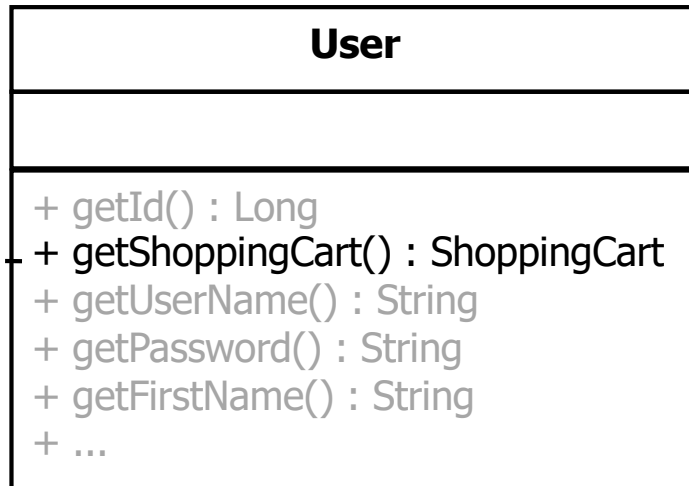
Modelado de relaciones - opcionalidad

- En el ejemplo anterior, en `OrderItem`, se utiliza **`optional=false`** en `@ManyToOne` sobre `getOrder`
 - Indica que `getOrder` nunca puede devolver `null`
 - La cardinalidad de `order` en la relación es 1 (cada línea de pedido está asociada obligatoriamente a un pedido)
 - Si `optional` fuese `true` (valor por defecto), `getOrder` podría devolver `null`
 - La cardinalidad de `order` en la relación sería 0..1
- `Order.getItems`
 - `@OneToMany` no dispone de `optional`
 - Si el pedido no tiene líneas de pedido asociadas (en este caso, no tiene sentido), devolvería un conjunto vacío (y NO `null`)

Modelado de relaciones 1:1 – User-ShoppingCart

Lado inverso

Lado propietario



`@OneToOne(mappedBy="user",
optional=false, fetch=FetchType.LAZY)`

`@OneToOne(optional=false, fetch=FetchType.LAZY)
@JoinColumn(name="userId")`

PK

id	userName	password	firstName	...
----	----------	----------	-----------	-----

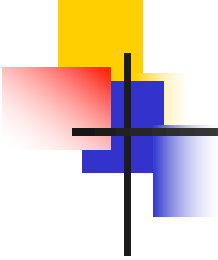
Tabla User

Tabla ShoppingCart

id	userId
----	--------

PK

FK



Modelado de relaciones 1:1 – User-ShoppingCart

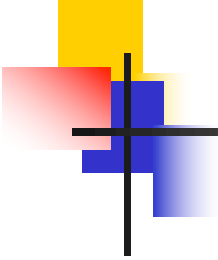
```
@Entity
public class User {

    ...
    private ShoppingCart shoppingCart;

    @OneToOne(mappedBy="user", optional=false, fetch=FetchType.LAZY)
    public ShoppingCart getShoppingCart() {
        return shoppingCart;
    }

    public void setShoppingCart(ShoppingCart shoppingCart) {
        this.shoppingCart = shoppingCart;
    }

    ...
}
```



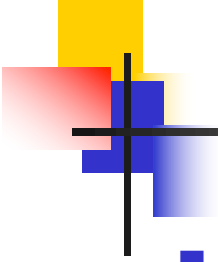
Modelado de relaciones 1:1 – User-ShoppingCart

```
@Entity
public class ShoppingCart {

    ...
    private User user;

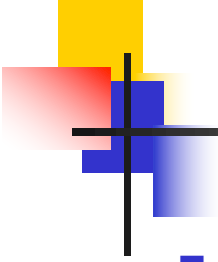
    @OneToOne(optional=false, fetch=FetchType.LAZY)
    @JoinColumn(name= "userId")
    public User getUser() {
        return user;
    }

    public void setUser(User user) {
        this.user = user;
    }
    ...
}
```



Modelado de relaciones 1:1

- Como en el caso 1:N, cambiando **@OneToMany** y **@ManyToOne** por **@OneToOne** en ambos lados
- **@OneToOne** también permite especificar **optional**

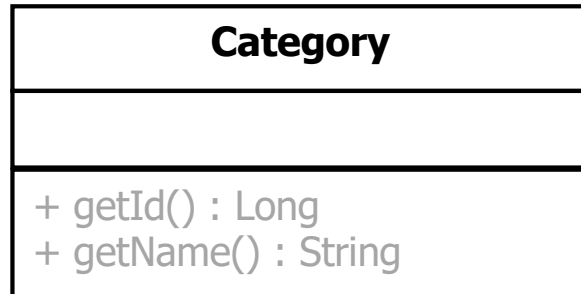


Relaciones unidireccionales

- A veces es conveniente modelar las relaciones como unidireccionales
 - Sólo se anota el método `get` del lado propietario (en las relaciones unidireccionales, no existe el lado inverso)
- Ejemplo: en Product-Category
 - Tiene sentido `product.getCategory()`
 - Pero no `category.getProducts()`, dado que podría devolver una gran cantidad de productos “de golpe” => limita la escalabilidad

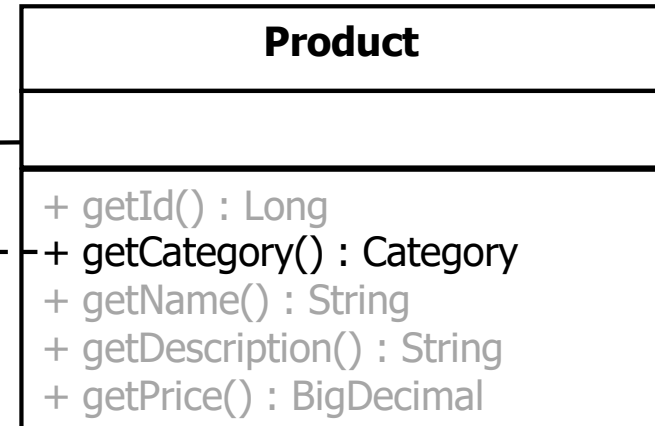
Relaciones unidireccionales

Lado propietario



1

1..n



```
@ManyToOne(optional=false, fetch=FetchType.LAZY)  
@JoinColumn(name="categoryId")
```

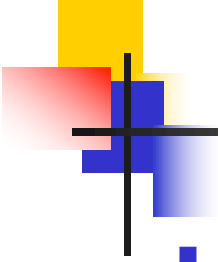
PK



Tabla Category

Tabla Product



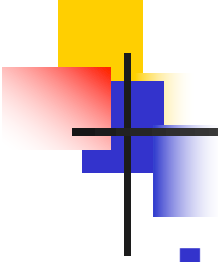


Modelado de relaciones – LAZY/EAGER

- Ejemplo: ¿qué ocurre cuando se le pide a un mapeador objeto-relacional que recupere una instancia de `OrderItem` de BD (por ejemplo, a partir de su id)

```
OrderItem item = entityManager.find(OrderItem.class, itemId);
```

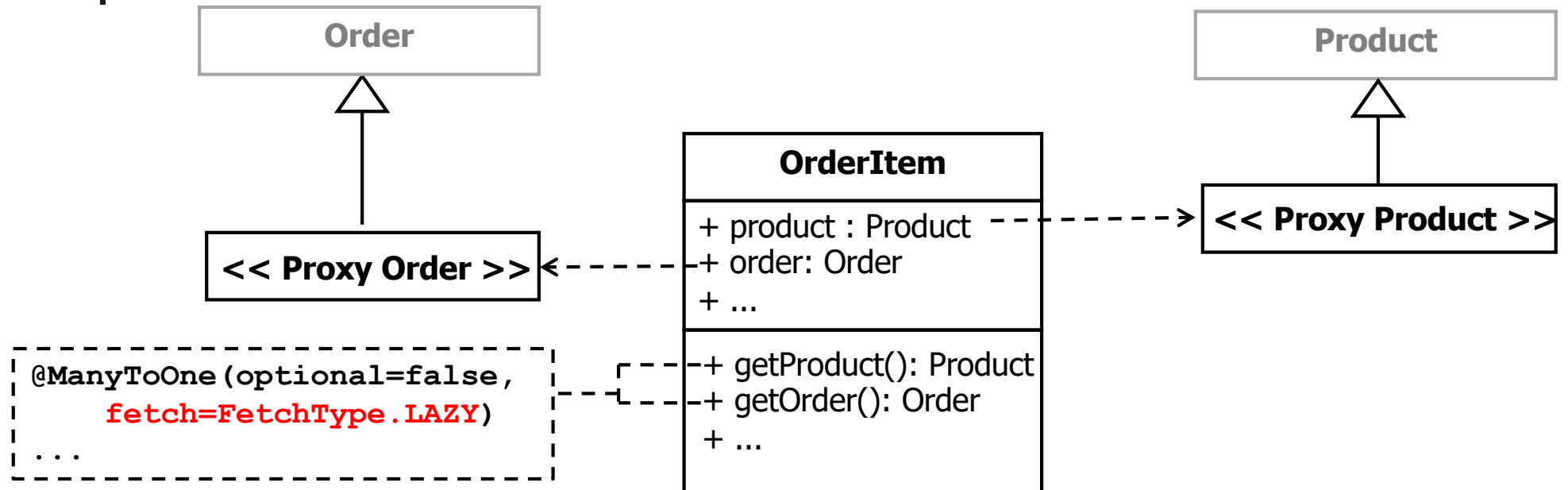
- Una implementación trivial haría
 - Recuperar la instancia de `OrderItem` (lanzando la correspondiente consulta `SELECT`)
 - Como `OrderItem` contiene los atributos `product` y `order`, también tendría que recuperar el producto y el pedido asociado
 - Como `Product` contiene el atributo `category`, tendría que recuperar la categoría asociada
 - Como `Order` contiene los atributos `items` y `user`, tendría que recuperar todos los ítems del pedido y el usuario asociado
 - Y de la misma forma, por cada ítem del pedido, y el propio usuario, habría que recuperar recursivamente las entidades relacionadas



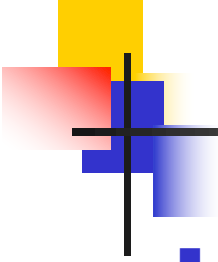
Modelado de relaciones – LAZY/EAGER

- Con JPA, por defecto, si una entidad tiene una relación **@xxxToOne** con otra, cuando se recupera la primera de BD, también se recupera la instancia relacionada de la segunda (si no estaba cargada ya en memoria)
 - Política **EAGER**
- En general, para evitar que se produzca el problema ilustrado anteriormente, se aconseja usar la política **LAZY** mediante el parámetro **fetch**
 - Cuando se recupera una instancia de una entidad de BD (e.g. **OrderItem**), los atributos (e.g. **product** y **order**) correspondientes a las propiedades anotadas con **@xxxToOne** se inicializan con proxies de las entidades correspondientes
 - Inicialmente el proxy sólo contiene la clave
 - Cuando se invoca una operación por primera vez sobre el proxy distinta a **get<<Clave>>**, el proxy lanza una consulta para recuperar el valor del resto de atributos

Modelado de relaciones – LAZY/EAGER



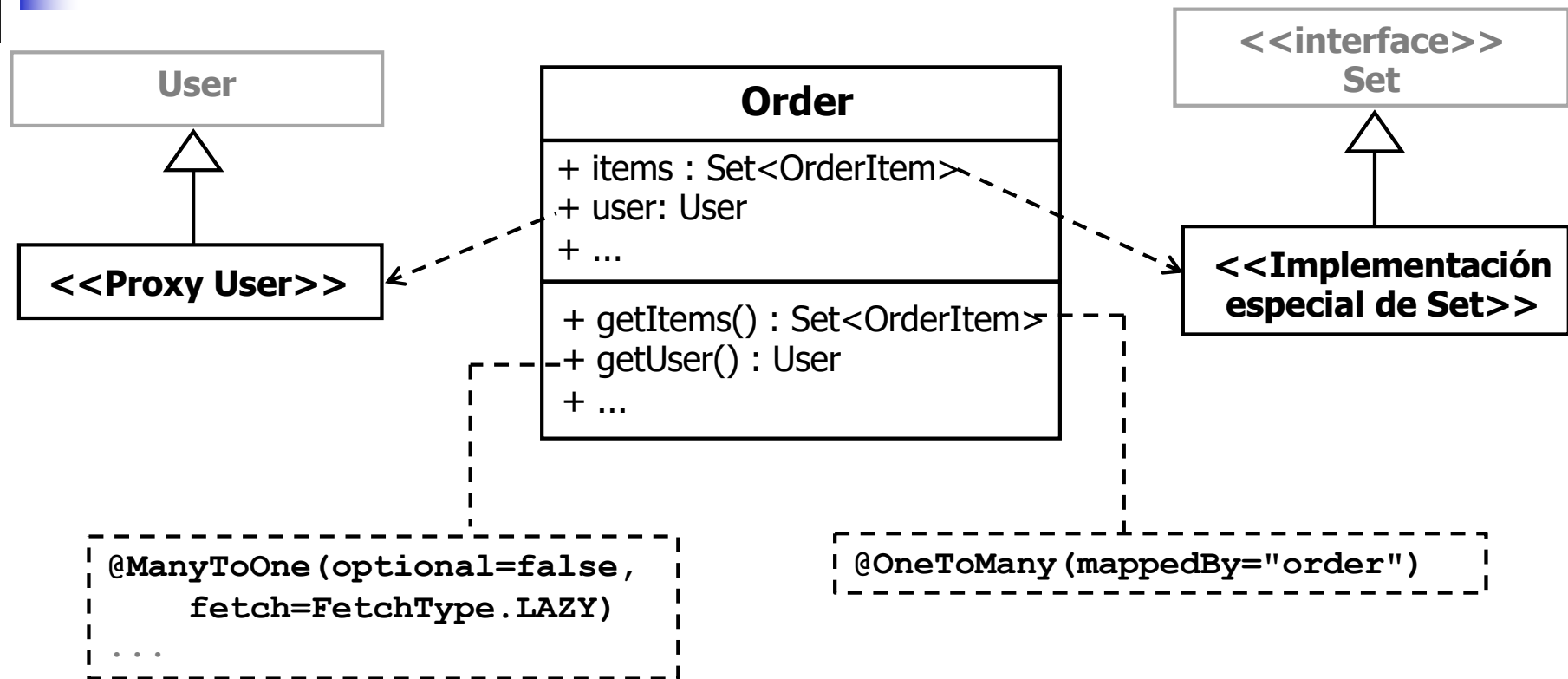
```
OrderItem item = entityManager.find(OrderItem.class,
    itemId); // SELECT * FROM OrderItem WHERE id=?
Order order = item.getOrder();
Long orderId = order.getId();
LocalDateTime date = order.getDate(); // SELECT * FROM OrderTable WHERE id=?
String postalAddress = order.getPostalAddress();
```



Modelado de relaciones – LAZY/EAGER

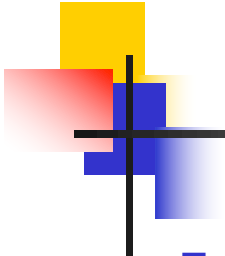
- Con JPA, por defecto, si una entidad tiene una relación `@xxxToMany` con otra, cuando se recupera la primera de BD, no se recuperan las instancias relacionadas de la segunda
 - Política **LAZY**
 - En este caso, el valor por defecto contribuye a que no se produzca el problema ilustrado anteriormente
 - Cuando se recupera una instancia de una entidad de BD (e.g. `Order`), los atributos (e.g. `items`) correspondientes a las propiedades anotadas con `@xxxToMany` se inicializan con una implementación especial de la colección (`Set`, en nuestro caso)
 - Cuando se invoca una operación, o se itera, por primera vez sobre esa implementación de la colección, se lanza una consulta para recuperar los valores de las instancias relacionadas

Modelado de relaciones – LAZY/EAGER



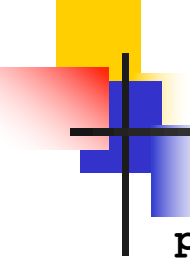
```
Order order = entityManager.find(
    Order.class, orderId); // SELECT * FROM OrderTable WHERE id=?
Set<OrderItem> items = order.getItems();

for (OrderItem item : items) { // SELECT * FROM OrderItem WHERE orderId=?
    process(item);
}
```



Métodos de negocio en entidades

- Además de los métodos **get/set** de las propiedades persistentes, las entidades pueden proporcionar métodos de negocio que faciliten la implementación de capas superiores: Lógica de Negocio (tema 3) y Servicios (tema 4)
 - Patrón Domain Model
 - Insistiremos en este aspecto en los temas 3 y 4
- Si el nombre de uno de estos métodos empieza por **get/is**, es necesario decirle al mapeador que este método no corresponde a una propiedad persistente
 - Se anota con **@Transient**



Métodos de negocio en entidades - Ejemplo

```
public class ShoppingCartItem {
```

```
...
```

```
public void incrementQuantity(int increment)
    throws MaxQuantityExceededException {
```

```
    if (quantity + increment > MAX_QUANTITY) {
        throw new MaxQuantityExceededException(
            MAX_QUANTITY - quantity);
    }
```

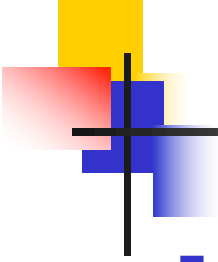
```
    this.quantity += increment;
```

```
}
```

```
@Transient
```

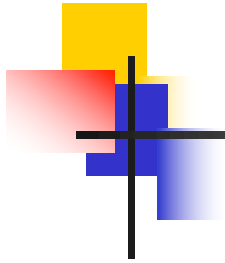
```
public BigDecimal getTotalPrice() {
    return product.getPrice().multiply(new BigDecimal(quantity));
}
```

```
}
```



Desarrollo de DAOs

- Spring Data permite desarrollar ágilmente DAOs para distintos tipos de fuentes de datos (BD relacionales con JDBC o JPA, NoSQL, servicios REST, etc.)
 - <https://spring.io/projects/spring-data>
- En particular, usaremos el soporte de Spring Data para JPA
 - <https://spring.io/projects/spring-data-jpa>
- Las interfaces de los DAOs no revelan ningún detalle de implementación
- En la mayor parte de los casos, Spring Data implementa automáticamente los métodos de los DAOs en tiempo de ejecución



Desarrollo de DAOs – operaciones CRUD

```
package es.udc.pashop.backend.model.entities;
```

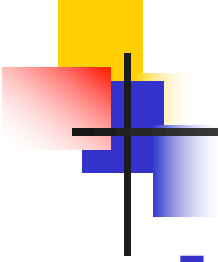
```
import org.springframework.data.repository.CrudRepository;
```

```
public interface OrderItemDao extends CrudRepository<OrderItem, Long>
{}
```

Tipo clave

Tipo entidad

- Todos nuestros DAOs extenderán de **CrudRepository**. Define:
 - Operaciones CRUD
- Algunos DAOs podrían necesitar extender adicionalmente de otras interfaces
 - Ejemplo: **CategoryDao** y su uso en **CatalogServiceImpl** (ver código fuente en el caso de estudio)



Desarrollo de DAOs – operaciones CRUD

- Ejemplo: buscar por id

```
Long id = ...;
Optional<OrderItem> item = orderItemDao.findById(id);

if (!item.isPresent()) {
    throw ...
}

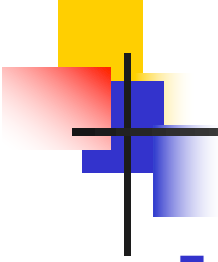
process(item.get());
```

- Ejemplo: añadir

```
OrderItem item = new OrderItem(...);

orderItemDao.save(item);
```

- NOTA: también se puede usar para actualizar, pero como veremos en el tema 3 no es necesario



Desarrollo de DAOs – operaciones CRUD

- Ejemplo: eliminar

```
Long id = ...;
Optional<OrderItem> item = orderItemDao.findById(id);

if (!item.isPresent()) {
    throw ...
}

orderItemDao.delete(item.get());
```

■ Es posible definir métodos de búsqueda usando convenciones de nombrado

- Veremos algunos ejemplos con los prefijos **findBy** y **existsBy**
- Después de ambos prefijos se expresa una condición en términos de propiedades de entidades, que siempre se escribe en CamelCase

■ Ejemplo 1

```
public interface UserDao extends CrudRepository<User, Long> {  
  
    boolean existsByUserName(String userName);  
  
    Optional<User> findByUserName(String userName);  
  
}
```

- En ambos casos la condición es muy sencilla: concuerda la instancia cuyo atributo/propiedad **userName** sea igual a la que se recibe en el primer parámetro
 - Obsérvese que **userName** empiece por mayúscula en el nombre de los métodos para respetar la notación CamelCase
- **findBy**
 - Se puede usar **optional** si se sabe que sólo puede devolver una instancia como máximo

- Ejemplo 2 (no pertenece al caso de estudio)
 - NOTA: la entidad `User` tiene que tener los atributos/propiedades `firstName` y `lastName`
 - Buscar todos los usuarios con un determinado nombre y apellidos

```
List<User> findByFirstNameAndLastName(String firstName,  
String lastName);
```
 - Buscar todos los usuarios con un determinado nombre o apellidos

```
List<User> findByFirstNameOrLastName(String firstName,  
String lastName);
```
 - Buscar todos los usuarios con un determinado nombre, ordenados por apellidos en orden ascendente

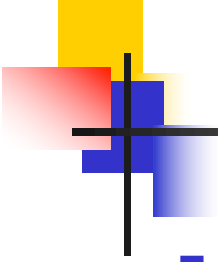
```
List<User> findByFirstNameOrderByLastNameAsc(  
String firstName);
```
 - Buscar todos los usuarios con un determinado nombre, ordenados por apellidos en orden descendente

```
List<User> findByFirstNameOrderByLastNameDesc(  
String firstName);
```

- Ejemplo 3: búsqueda de los pedidos de un usuario. Un primer enfoque

```
public interface OrderDao extends CrudRepository<Order, Long> {  
  
    List<Order> findByUserIdOrderByDateDesc(Long userId) ;  
}
```

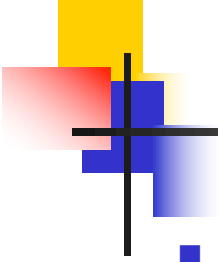
- Encontrar los pedidos de un usuario, ordenados por fecha en orden descendente
- **UserId** => navegación entre entidades
 - `order` no tiene el atributo/propiedad `userId`
 - Tiene `user`, de tipo `User`
 - `User` tiene el atributo/propiedad `id`
 - En consecuencia **UserId** se refiere al valor `order.getUser().getId()`
- **Date**
 - `order` tiene el atributo/propiedad `date`



Desarrollo de DAOs – paginación

■ Ejemplo 3

- Observación: un usuario, con el tiempo, puede haber hecho un gran número de pedidos
- Se limita la escalabilidad
 - Muchos usuarios pueden invocar, quizás concurrentemente, el caso de uso de búsqueda de pedidos
 - La consultas que recuperan los pedidos de cada usuario traen gran cantidad de datos de la BD al servicio
 - Se crean listas grandes en memoria dentro del servicio con los datos de cada consulta
- Se limita la usabilidad
 - La IU mostraría los datos de todos los pedidos del usuario, en lugar de mostrarlos paginados (a los que el usuario accedería bajo demanda)



Desarrollo de DAOs – paginación

- Ejemplo 3

- Solución: **paginación** => recuperar los datos “paginados” (en bloques), bajo demanda

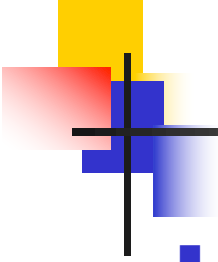
```
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Slice;
import org.springframework.data.repository.CrudRepository;

public interface OrderDao extends CrudRepository<Order, Long> {

    Slice<Order> findByUserIdOrderByDateDesc(
        Long userId, Pageable pageable);

}
```

- Los métodos de búsqueda pueden
 - Recibir un parámetro de tipo **Pageable**: especifica el rango de resultados que se quieren recuperar de la BD
 - Devolver un valor de tipo **slice**: contiene los datos solicitados y una indicación de si hay más



Desarrollo de DAOs – paginación

- Ejemplo 3
 - Invocación

```
import org.springframework.data.domain.PageRequest;
```

```
...
```

```
int page = ...;
```

```
int size = ...;
```

```
Slice<Order> slice = orderDao.findByUserIdOrderByDateDesc(  
    userId, PageRequest.of(page, size));
```

```
List orders = slice.getContent();
```

```
boolean existMoreOrders = slice.hasNext();
```

Desarrollo de DAOs – paginación

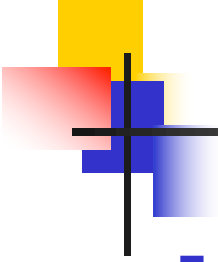
■ Ejemplo 3

Purchase order	Date
13	6/3/2025 - 1:30 PM
12	6/3/2025 - 1:30 PM
11	6/3/2025 - 1:29 PM
10	6/3/2025 - 1:29 PM
9	6/3/2025 - 1:29 PM
8	6/3/2025 - 1:28 PM
7	6/3/2025 - 1:28 PM
6	6/3/2025 - 1:28 PM
5	6/3/2025 - 1:25 PM
4	6/3/2025 - 1:25 PM
Back Next	

page=0, size=10,
slice.getContent() contiene 10 pedidos
slice.hasNext() devuelve true

Purchase order	Date
3	5/30/2025 - 1:15 PM
2	5/12/2025 - 11:40 AM
1	5/9/2025 - 11:04 AM
Back Next	

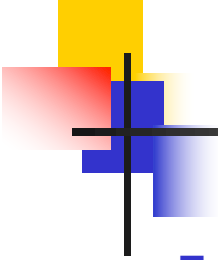
page=1, size=10,
slice.getContent() contiene 3 pedidos
slice.hasNext() devuelve false



Desarrollo de DAOs – paginación

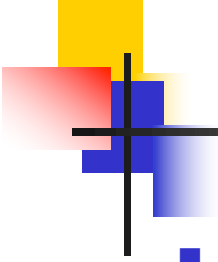
- Ejemplo 3. Observaciones

- Para que la paginación produzca una visualización de datos consistente entre páginas, es fundamental que la consulta devuelva los datos ordenados según algún criterio (e.g. el sufijo `OrderByDateDesc` en el ejemplo)
- La implementación usa la API de JDBC para informar a la BD que no devuelva todas las filas que concuerdan con la búsqueda, sino sólo aquellas que están en el rango solicitado: `[page*size, (page*size)+size-1]`



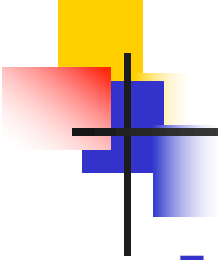
Desarrollo de DAOs – JPQL

- La definición de métodos de búsqueda mediante convenciones de nombrado es una opción cómoda
- Pero ...
 - No es conveniente cuando el nombre del método queda demasiado grande
 - No es útil cuando no hay forma de expresar con convenciones de nombrado la condición de la búsqueda
- JPA permite realizar consultas mediante
 - JPQL (Java Persistence Query Language): lenguaje declarativo
 - La API Criteria: permite expresar una consulta mediante objetos
- Estudiaremos lo esencial de JPQL



Desarrollo de DAOs – JPQL

- JPQL tiene intencionadamente una sintaxis parecida a SQL, de forma que sea sencillo de aprender
- ¡Pero no es SQL!
 - La sintaxis es similar, pero no igual
 - Se usan nombres de entidades y atributos/propiedades en lugar de tablas y columnas
- Ejemplo: encontrar un usuario a partir de su **userName**
 - **SELECT u FROM User u WHERE u.userName = :userName**
 - **:userName** es un “parámetro nombrado”: la API de JPA permite sustituirlo por un valor antes de lanzar la consulta
- Cuando se lanza una consulta JPQL, la implementación de JPA
 - Averigua el nombre de la tabla y las columnas correspondientes a los atributos/propiedades (e.g. anotaciones, ficheros de configuración)
 - Traduce la consulta JPQL a una consulta SQL
 - La ejecuta vía JDBC
 - Recupera las filas y las devuelve como instancias de entidades



Desarrollo de DAOs – JPQL

- Más ejemplos

- Equivalentes a los métodos de búsqueda definidos anteriormente

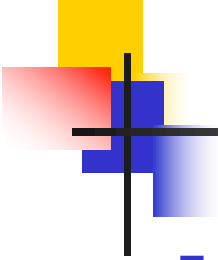
- ```
SELECT u FROM User u WHERE u.firstName = :firstName AND
u.lastName = :lastName
```

- ```
SELECT u FROM User u WHERE u.firstName = :firstName OR  
u.lastName = :lastName
```

- ```
SELECT u FROM User u WHERE u.firstName = :firstName ORDER BY
u.lastName ASC
```

- ```
SELECT u FROM User u WHERE u.firstName = :firstName ORDER BY  
u.lastName DESC
```

- ```
SELECT o FROM Order o WHERE o.user.id = :userId ORDER BY o.date
DESC
```



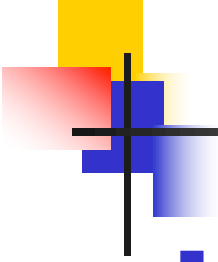
# Desarrollo de DAOs – JPQL

- Spring Data JPA permite lanzar consultas con JPQL de manera sencilla mediante la anotación `@Query` en los métodos de los DAOs
  - `org.springframework.data.jpa.repository.Query`
- Ejemplos
  - Se usa `?i` para referirse la parámetro “i” del método (de 1 en adelante)

```
@Query("SELECT u FROM User u WHERE u.userName = ?1")
Optional<User> findUser(String userName);
```

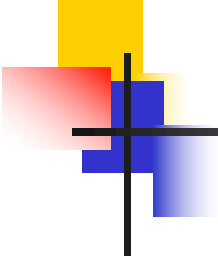
```
@Query("SELECT u FROM User u WHERE u.firstName = ?1 OR u.lastName = ?2")
List<User> findUsers(String firstName, String lastName) // Sin paginación
```

```
@Query("SELECT o FROM Order o WHERE o.user.id = ?1 ORDER BY o.date DESC")
Slice<Order> findOrders(Long userId, Pageable pageable); // Con paginación
```



# Desarrollo de DAOs – JPQL

- La anotación `@Query` no es válida cuando hay que generar dinámicamente la consulta
- Ejemplo: búsqueda de productos por palabras clave y categoría en el caso de estudio
- En estos casos es necesario
  - 1: Definir este tipo de operaciones de búsqueda en una interfaz adicional
    - `CustomizedProductDao`
  - 2: Implementarla
    - `CustomizedProductDaoImpl`
    - Hay que hacer uso de la API de JPA para lanzar la consulta
  - 3: Hacer que la interfaz del DAO extienda la interfaz adicional
    - `ProductDao` extiende `CustomizedProductDao`



# Desarrollo de DAOs – JPQL

---

- ① 

```
public interface CustomizedProductDao {

 Slice<Product> find(Long categoryId, String keywords, int page,
 int size);

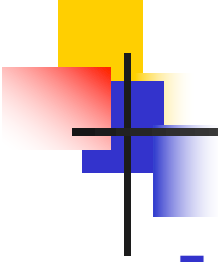
}
```
- ② 

```
public class CustomizedProductDaoImpl implements CustomizedProductDao {

 // Ver código fuente en el caso de estudio

}
```
- ③ 

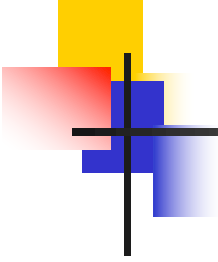
```
public interface ProductDao extends
 CrudRepository<Product, Long>, CustomizedProductDao {}
```



# Configuración

---

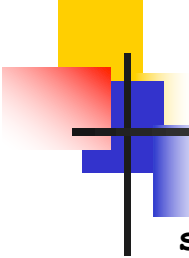
- Con Spring Boot la configuración reside en `src/main/resources/application.{properties,yml}`
- En lo que respecta a la capa de Acceso a Datos, se configura
  - El pool de conexiones
  - Hibernate (e.g. que no genere las tablas)
- La configuración puede estar en formato `properties` o YAML (YAML Ain't Markup Language)
  - El caso de estudio usa YAML: <https://yaml.org>



# Configuración – formato properties

En el código fuente de PA Shop se usan los valores `@dataSource.url@`, `@dataSource.user@` y `@dataSource.password@` para que Maven los sustituya (fase `process-resources`) por los valores especificados en las propiedades `dataSource.url`, `dataSource.user` y `dataSource.password` definidas en el `pom.xml`.

```
spring.datasource.url=@dataSource.url@
spring.datasource.username=@dataSource.user@
spring.datasource.password=@dataSource.password@
spring.datasource.hikari.maximum-pool-size=4
spring.jpa.hibernate.ddl-auto=none
spring.jpa.hibernate.naming.physical-strategy=...
...
```



# Configuración – formato YAML

```
spring:
 datasource:
 url: @dataSource.url@
 username: @dataSource.user@
 password: @dataSource.password@
 hikari:
 maximum-pool-size: 4
 jpa:
 hibernate:
 ddl-auto: none
 naming:
 physical-strategy: ...
 ...
```



Ojo: tienen que ser blancos, no valen tabuladores